

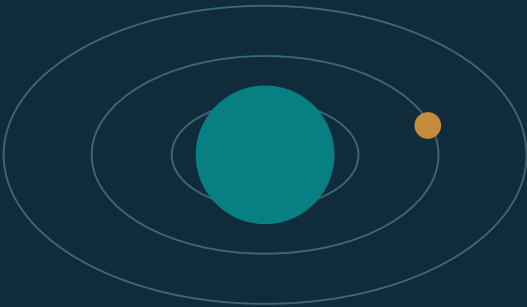
FIELD GUIDE / 01

Astroboat interview field guide

A beginner-to-confident guide for your club project showcase.

- KNOW THE PRODUCT.
- TRACE THE DATA.
- EXPLAIN THE TRADEOFFS.

Your goal: explain clearly, then go one layer deeper.



THE ONE-SENTENCE EXPLANATION

Astroboat turns scattered astronomy information into one approachable website: imagery, briefs, events, Moon data, asteroid approaches, and an astronomy assistant.

Use this guide in three passes

Pass	Read	Outcome
1 / Understand	Pages 2-4	Explain the problem, stack, and architecture.
2 / Go deeper	Pages 5-12	Trace real features and explain the engineering.
3 / Rehearse	Pages 13-17	Answer questions, demonstrate, and revise quickly.

If the interview is soon: read pages 2, 4, 9, 15, and 17 first. Then practice the demo on page 16 out loud.

Prepared for Khushal Dangar | Code reviewed: 29 September 2026
Based on the local astrobost-v2 working tree. Current code takes precedence over older README claims. Live website availability and production configuration were not verified.

FIELD GUIDE / 02

What you are showcasing

Start with the user problem, then explain the product.

The problem and the value

Astronomy information is spread across observatory websites, news feeds, research papers, and specialized APIs. A curious beginner has to visit several places and interpret unfamiliar terms. Astroboat gathers that material, gives it a consistent shape, and presents it with context and original-source links.

YOUR 45-SECOND OPENING

"Astroboat is an astronomy discovery platform. It combines imagery, short briefs, launch and sky events, Moon information, and asteroid close approaches. The app uses Next.js and React, with server-side services that normalize several public sources. It also has a Groq-powered assistant for explanations. A key focus is keeping the interface useful when a provider fails and making the source of the information visible."

Feature	What to show
Discover + Gallery	Interactive 3D Earth; astronomy image search, filters, and lightbox.
Briefs + Events	Short source-linked updates; a chronological launch and sky-event list.
Moon + Asteroid Watch	Phase and rise/set data; close approaches with readable units.
Ask Astroboat	Astronomy explanations through a server-side AI API route.

The honest scope

The project has no application database, account system, saved-user-content feature, or model training pipeline. Articles, Learn, and Satellites currently show paused-feature notices. The standalone Cloud Run/Vertex AI example and root-level HTML/JavaScript prototype are separate from the active Next.js app.

Strong answer to "What is original here?" The contribution is the integration, data cleanup, presentation, interaction design, and failure handling. The astronomy observations and language model come from external providers. Describe your personal contribution accurately; use the ownership exercise on page 16.

FIELD GUIDE / 03

The stack, in plain English

Learn what each tool contributes rather than memorizing a list of names.

Technology	Meaning in Astroboat
React 19.1	Builds the interface from reusable components. State changes update what the user sees.
Next.js 15.5	Provides file-based routes, server rendering, API handlers, and fetch revalidation around React.
TypeScript 5.7	Checks expected data shapes during development. Strict mode helps catch mistakes before runtime.
Tailwind CSS 3.4	Utility classes style layouts and components; custom CSS adds the atmosphere and visual identity.
fast-xml-parser	Turns RSS/Atom XML feeds into JavaScript data that the Briefs and Gallery services can process.
Three.js	Builds the homepage Earth scene using WebGL, textures, a camera, materials, and shaders.
Groq + a hosted model	Provides generated assistant answers. The code defaults to llama-3.3-70b-versatile; an environment setting can override it.

Six terms you should be able to explain

- API:** a program-to-program interface. Astroboat requests data and receives a response.
- JSON:** structured data using objects and arrays; common in the astronomy APIs.
- RSS/Atom:** XML feed formats publishers use to distribute updates.
- Component:** a reusable part of the interface, such as a card or filter bar.
- Props:** data passed into a component by its parent.
- State:** changing local values, such as the active filter or typed chat message.

A GOOD FOLLOW-UP ANSWER

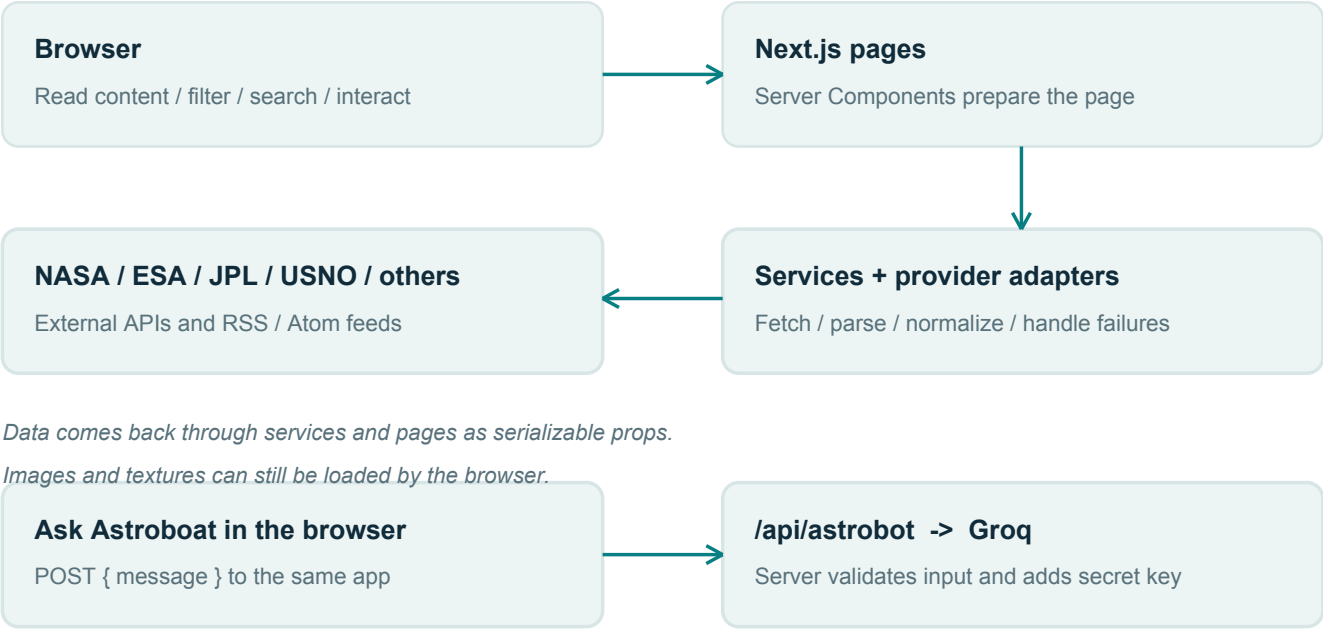
"TypeScript helps me use data correctly, but a TypeScript type does not validate an external response at runtime. The services also need checks for missing fields, invalid numbers, and unexpected formats."

Version numbers above reflect package.json declarations in the reviewed code. They describe this project, not a claim about the latest available releases.

FIELD GUIDE / 04

The architecture you should draw

Two paths: astronomy data for pages, and AI requests from chat.



The main request path

A page such as `/briefs` runs an async Server Component. It calls a service, receives a normalized result, and passes that result into an interactive Client Component. The browser then handles search and filtering against that supplied list. The page does not need to call its own API route to use a server service.

Server Components and client islands

Server Components can fetch data on the server and keep provider credentials there. Components marked **"use client"** support browser interaction, React state, and effects. They can still have initial HTML rendered on the server; the directive does not mean "nothing is server-rendered." Hydration makes the client interface interactive in the browser.

Folder	Responsibility
app/	URL routes, layouts, metadata, and API route handlers.
services/	Provider requests, parsing, normalization, calculations, and fallbacks.
components/	Cards, navigation, feature interfaces, and visuals.
lib/	Shared types, constants, helper functions, and sample fallback data.

Memory hook: pages coordinate, services translate, components present. Source/provider fetching is server-side; browsers still fetch image assets and local textures.

FIELD GUIDE / 05

Deep dive: Astronomy Briefs

Your best feature for explaining a complete data pipeline.

Walk one story through the system

- 1. Fetch:** `getAstronomyBriefs` selects nine configured feeds and starts them concurrently with `Promise.all`. Each feed function catches its own errors and returns a status, so a failed feed normally does not reject the entire batch.
- 2. Parse and normalize:** the XML parser handles RSS/Atom structure. The service extracts a title, original URL, date, description, tags, and image into a shared `AstronomyBrief` shape. Missing essential title/URL data causes an item to be discarded.
- 3. Enrich:** when artwork is missing, the service can inspect trusted publisher pages and image metadata. Some enrichment work uses bounded concurrency to avoid launching too many requests at once.
- 4. Clean and summarize:** text is stripped of markup, the first two sentences are selected, and the result is shortened to about 360 characters. Tags and "why it matters" text come from rules and category templates. These briefs are **not AI-generated summaries**.
- 5. Merge:** normalized URLs and titles go into Sets to remove duplicates. The combined list is sorted newest first and limited to 40 items. The UI searches and filters that list locally.

```
const result = await getAstronomyBriefs();
return <BriefsClient result={result} />;

// Simplified flow:
fetch -> parse -> enrich -> normalize -> dedupe -> sort -> UI
```

Why normalization matters

One source may call its text "description," another "summary." The interface should not need to know which provider supplied the story. A shared model isolates provider differences and makes cards easier to reuse.

INTERVIEW ANGLE

"The difficult part is handling inconsistent feeds and partial failures, not just making a fetch call. I can explain where raw data becomes a predictable UI model and why healthy sources remain useful when others fail."

Know the limit: detail pages look up stories in the current feed set or bundled samples. There is no persistent article archive. IDs also include feed position, so permalink stability is a useful improvement to discuss.

FIELD GUIDE / 06

Deep dive: Astronomy Gallery

Data quality and image behavior matter as much as visual design.

How the collection is assembled

The service requests NASA APOD, APOD RSS, NASA Image Library searches, and ESA/Webb, Hubble, and ESO feeds. It uses **Promise.allSettled** so it can inspect successful and failed provider results independently. APOD API and RSS requests are started together in the current code.

Candidates are scored using titles, descriptions, sources, and other metadata. Keyword rules aim to reject portraits, hardware, illustrations, and irrelevant media while keeping astronomy observations. The merged results are deduplicated and ranked using freshness bands, relevance score, and date, with a cap of 120 images.

BE PRECISE ABOUT THE INTELLIGENCE

"The filtering is a heuristic based on metadata. It is not a trained image-classification model, so false positives and false negatives are possible."

What happens in the browser

GalleryClient receives the initial dataset as props. React state stores the selected category, search text, visible count, and selected image. useMemo derives the filtered list. It initially reveals 24 images and adds 18 on each Load more click. This is **client-side progressive reveal**; that button does not fetch a new provider page.

Selecting an image updates a query parameter so the lightbox can be linked. This link depends on the image remaining in the available dataset. Image components try candidate image renditions and keep attribution available when loading fails.

There is also an API route

GET /api/gallery supports an image ID or query/category/page/limit parameters, with a maximum limit of 60. The active GalleryClient does its ordinary search and Load more locally instead of using that route for every interaction.

A thoughtful fallback choice

The curated fallback gallery is currently empty. If all providers are unavailable, the service returns no substitute photos. This avoids attaching astronomy captions to unrelated images. Provider warnings are collected, although the gallery page does not currently surface every partial-source warning.

Tradeoff: client filtering is responsive for a bounded list. A much larger archive would need server pagination or search so the initial response and browser workload stay manageable.

FIELD GUIDE / 07

Moon and events: time has context

Be ready to explain source, location, time zone, and freshness.

Moon: information for a specific observer

The Moon service requests daily data and upcoming primary phases from the **U.S. Naval Observatory (USNO)** in parallel. It normalizes phase names, illumination, moonrise, moonset, and phase dates, then adds rule-based observing advice. The SVG Moon is a visualization of those values; it is not a live telescope image.

The page currently uses **Ahmedabad, India: 23.0225, 72.5714, UTC+05:30**. The service accepts other coordinates and an offset, but the active page does not provide a location picker. Rise/set times depend on location; primary phase events are presented in UTC.

THE ASTRONOMY QUESTION

Moon phases happen because we see different portions of the Moon's sunlit half as it orbits Earth. Ordinary phases are not caused by Earth's shadow. "Waxing" means the visible illuminated fraction is increasing; "waning" means it is decreasing. A quarter Moon appears roughly half illuminated.

Astronomy reference: [NASA - Moon Phases](#).

Events: launches, missions, and curated sky phenomena

The Events service gets launches and spaceflight events from **The Space Devs Launch Library 2**, adds curated sky events, removes duplicates, sorts by date, and limits the result. The interface is a filtered chronological list, even though some names in the code say "calendar."

The combined service checks launches first. If it detects throttling, it skips another upstream event call and supplies fallback events. A process-local cooldown avoids immediately repeating calls to a throttled provider. It is not a shared rate limiter across every server instance.

The subtle caveats

A launch schedule can change, and a six-hour cache is not a real-time stream. The Moon service can mix a live daily response with saved phase events without setting the overall fallback flag. That means the UI may not disclose every partial fallback. A good next step is per-dataset source and freshness status.

FIELD GUIDE / 08

Asteroid Watch: explain the numbers

Present close-approach information without claiming impact prediction.

The provider chain

JPL CAD first: query Earth close approaches over the next 30 days, within 0.2 AU, with a maximum of 20 results. **NASA NeoWs next:** if JPL yields no objects and NASA_API_KEY exists, request the service's seven-day date span. **Bundled sample data last:** used if neither provider yields results. These providers have different query coverage; the fallback is not an identical replacement.

Why the JPL mapper uses field names

JPL returns a fields array and rows of values. The mapper finds columns such as **des** (designation), **dist** (distance), and **v_rel** (relative speed), instead of assuming a fixed column order. It rejects rows missing essential data. JPL documents dist in AU and v_rel in km/s.

```
distanceKm = distanceAu * 149597870.7
distanceLunar = distanceKm / 384400
speedKph = speedKps * 3600
```

```
Example only: 0.01 AU = about 1,495,979 km
                = about 3.89 lunar distances
                20 km/s    = 72,000 km/h
```

Unit reference: [JPL - SBDB Close Approach Data API](#). The numeric example above is illustrative, not a current object.

Size estimates are estimates

The code estimates diameter from absolute magnitude H using $D(\text{km}) = 1329 / \sqrt{\text{albedo}} \times 10^{(-H/5)}$. It assumes albedos of 0.25 and 0.05 to produce a range. Albedo describes reflectivity: a darker object may need to be larger to produce the same brightness. This is not a measured diameter.

The labels need careful explanation

The app maps a supplied potentially-hazardous flag to "watch," distance at or below five lunar distances to "notable," and larger distances to "safe." These are UI heuristics. The JPL mapping currently sets the hazard flag to false rather than fetching that classification, so false should not be interpreted as confirmed absence of hazard.

ANSWER TO "CAN IT PREDICT AN IMPACT?"

"Astroboat displays provider close-approach data and educational context. It does not calculate collision probability. I would improve unknown hazard handling and wording before treating its labels as scientific risk assessments."

Ask Astroboat: the exact AI flow

An API integration with guardrails, not a model trained by this project.

- 1. AstrobotClient stores visible messages, input, loading state, and errors in React state.
- 2. It sends POST /api/astrobot with a JSON body containing only message.
- 3. The route parses JSON, trims input, rejects empty/invalid input, and caps it at 1,000 characters.
- 4. The server reads GROQ_API_KEY and sends a system prompt plus the current user message to Groq.
- 5. It validates the returned answer and sends { answer } back, or a controlled error.

```
Browser -> POST /api/astrobot -> Groq -> answer -> UI

Model default: llama-3.3-70b-versatile
Controls: temperature 0.5 | max_tokens 500
Request timeout: 30 seconds | fetch cache: no-store
```

Question	Correct answer
Does it remember?	The UI shows the conversation, but earlier messages are not sent upstream. It is single-turn at the model layer.
Does it retrieve live facts?	No retrieval or tool calling is implemented. The prompt redirects current-data questions to Events, Moon, and Asteroid Watch.
Does it stream?	No. The browser waits for a complete JSON answer. The typing indicator is a loading state.
Where is the key?	On the server, in GROQ_API_KEY. It is added to the upstream Authorization header, not sent as client configuration.

Know the error codes

400: invalid input. **503:** missing configuration, provider authentication failure, or unavailable model. **429:** upstream request limit. **502:** provider/network/answer failure. **504:** timeout. The endpoint avoids exposing provider error messages or secrets to the browser.

Limits: the astronomy-only system prompt guides the model but cannot guarantee every response follows it. There is no local per-user rate limiter or authentication. A key in a server environment is necessary, but it does not stop abuse of an open endpoint.

The Earth scene and React hooks

A strong visual feature with useful performance decisions behind it.

How the 3D Earth works

Hero renders EarthScene. That client component waits until its container enters the viewport, then dynamically imports the Three.js implementation. The scene has a perspective camera, a sphere, day/night textures, a cloud layer, an atmospheric shader, and a decorative orbital ring.

A **texture** is an image mapped onto geometry. A **shader** is GPU code that determines how a surface is drawn. The Earth shader uses surface direction relative to a fixed light direction to blend daytime appearance and nighttime lights. The scene is an artistic illustration, not a physically current Earth day/night map.

Hook / API	How this project uses it
useState	Stores values that affect the interface: ready, failed, paused, filters, and messages.
useRef	Keeps a DOM node or scene controller available without triggering a render when the reference changes.
useEffect	Starts browser-only work, attaches listeners, and returns cleanup when the component unmounts.
useMemo	Reuses a derived filtered array until its dependencies change. It is an optimization, not permanent storage.
IntersectionObserver	Detects visibility to delay loading and pause offscreen rendering.
ResizeObserver	Updates renderer size and camera aspect ratio when the container dimensions change.

Performance and accessibility choices

The renderer caps pixel ratio at 1.75, asks for low-power rendering, and stops the animation loop when offscreen, hidden, or paused. Reduced-motion preference pauses auto-rotation. Users can drag or use rotation buttons. A CSS/image fallback supports loading or WebGL failure.

Cleanup removes listeners and observers, stops animation, and disposes textures, geometry, materials, and the renderer. Without cleanup, repeated mounting can leave GPU memory and event handlers behind.

THE TRADEOFF

"Three.js adds visual impact but also bundle and GPU cost. Lazy loading, visibility-aware rendering, reduced-motion support, and resource cleanup control that cost. I would measure actual performance before claiming a speed improvement."

Reliability and security

Explain what already exists and where protection is incomplete.

Data	Revalidation / failure behavior
Briefs	1 hour. Keep available feeds; bundled samples when no live briefs remain.
Gallery	6 hours. Keep usable provider results; empty result when all fail.
Events	6 hours. Source-specific samples and throttling warnings.
Moon	6 hours. Saved data on failure; partial phase fallback needs better labeling.
Asteroids	6 hours. JPL -> configured NeoWs -> bundled samples.
Assistant	No-store upstream request. Return an error rather than a fabricated local answer.

What caching actually promises

The services request time-based revalidation through Next.js fetch options. Cached responses reduce repeat provider calls and latency. The interval is a freshness policy, not a background timer or a promise that every browser updates exactly then. Provider failures and cache behavior can leave older information visible.

A **cache** reuses a fetched response. A **fallback** is replacement behavior when useful data is unavailable. Bundled samples are not automatically the last successful live response. An assembly timestamp such as `lastUpdated` is not necessarily the provider's publication time.

Existing protections

Assistant credentials stay server-side; input is checked again on the server; upstream errors are sanitized. Brief enrichment limits allowed hostnames, checks redirects, and bounds concurrent work. Publisher markup is normalized into text for ordinary rendering instead of inserting raw article HTML.

What you would strengthen

Add request limiting for chat, consistent timeouts for all providers, runtime response schemas, and explicit live/stale/sample status for each dataset. Audit every external URL and any JSON-LD script serialization path before claiming full input-safety coverage. The current controls are useful, but they are not a security audit.

REMEMBER THIS DEFINITION

"Graceful degradation means losing a provider should reduce the amount of available information, rather than make the whole experience unusable. Users should also be told when the data is incomplete or saved."

Testing, running, and deployment

Evidence you can discuss without inventing a production success story.

What was verified for this guide

Check	Result and boundary
npm run test:assistant	15 tests passed. Covers model names, missing keys, invalid input, provider errors, timeout behavior, and safe diagnostics.
npx tsc --noEmit --incremental false	Passed with no TypeScript errors. This checks types, not live provider behavior or visual correctness.
Repository review	Traced routes, client components, services, constants, assistant tests, and the Three.js scene.
Not verified here	Full production build, deployed configuration, current provider availability, browser end-to-end behavior, or measured performance.

Why mocked tests are useful

The existing test file uses Node's test runner, transpiles the route with TypeScript, and runs it with controlled environment and fetch behavior. It can simulate a 429 or timeout without real credentials, API cost, or a live outage. Passing mocked tests does not prove that the deployed key and model are available.

Local workflow

```
npm ci           # install locked dependencies
npm run dev      # local development
npm run test:assistant # deterministic route checks
npx tsc --noEmit # type check
npm run build    # production build
npm run start    # serve the built app
```

GROQ_API_KEY enables the assistant; GROQ_MODEL optionally changes its default. NASA_API_KEY enables the NeoWs fallback and can be used by APOD. NEXT_PUBLIC_SITE_URL supplies a public site origin. Keep secrets out of client-prefixed variables and source control. Restart the server after changing environment settings.

What deployment requires

Use a server-capable Next.js host with outbound provider access. A pure static file host cannot run the assistant route. Build-time work may need external data or font access. Hosted environment variables are separate from your local .env.local.

Next useful tests: malformed JPL rows and units; Moon partial failure; duplicate stories; one healthy gallery source; chat error UI; keyboard-only search and lightbox. Do not describe these as implemented tests.

S1, S11; commands executed during this review

FIELD GUIDE / 13

Practice questions: the fundamentals

Answer in 20-40 seconds, then point to one concrete example.

1. Why did you choose this project?

"It makes astronomy easier to explore while giving me a concrete engineering problem: bringing inconsistent public data into one understandable interface." Add your real motivation, such as curiosity about space or wanting to learn API integration.

2. Is this frontend or full stack?

"It includes both. React components provide the interface, while Next.js server pages, service modules, and API handlers fetch and process data. It has backend logic even though it does not have an application database."

3. Why Next.js instead of React alone?

"Next.js provides routing, server rendering, metadata, caching controls, and API routes in one project. That fits a content-heavy site with public data and a server-only AI key. React handles the component model and interaction."

4. What happens when I open Briefs?

"The route calls `getAstronomyBriefs` on the server. That function fetches feeds, cleans and normalizes items, merges duplicates, and returns a typed result. `BriefsClient` receives it as props and handles local search and filters."

5. Why is there no database?

"The current app mainly reads provider data and keeps temporary interface state. A database is not needed for that initial scope. I would introduce persistence for bookmarks, accounts, an article archive, or historical observations."

6. What is the difference between props and state here?

"Props are the dataset given to `GalleryClient`. State is what changes as the user interacts, such as the selected category or image. A derived filtered list combines the props with the state; I do not need to mutate the original dataset."

ACTIVE RECALL

After each answer, ask yourself: "Can I name the file or function that demonstrates this?" If not, revisit that section before memorizing the answer.

FIELD GUIDE / 14

Practice questions: engineering choices

These answers show understanding of cause, effect, and tradeoffs.

7. Why fetch data on the server?

"It keeps credentials and provider-specific parsing out of the browser and enables shared caching. It also gives the page usable initial data. The tradeoff is server work and possible waiting on slow providers, so timeout and fallback behavior matter."

8. Promise.all or Promise.allSettled?

"Promise.all rejects if a member rejects, but Briefs catches failures inside each source fetch and returns status objects. Gallery uses Promise.allSettled to inspect outcomes independently. Both support partial results here, through different error-handling structures."

9. How do you remove duplicate stories?

"Briefs normalizes URLs, removing tracking parameters, and normalizes titles. It keeps Sets of seen keys and skips repeats. Set-based lookup is typically efficient, but title rules are approximate and can merge distinct stories or miss rewritten duplicates."

10. Is the Load more button pagination?

"In the Gallery UI it reveals a larger slice of an already received array. That reduces the number of rendered cards initially, but it does not reduce the initial dataset transfer. The gallery API also supports slicing by page, but the UI does not use it for this button."

11. How would you scale it?

"First I would measure response times, provider failure rates, payload sizes, and rendering cost. Then I would separate expensive feed ingestion from page requests, persist a normalized archive, use shared cache/limits across instances, and paginate large datasets. Chat needs independent abuse and cost controls."

12. What is your strongest testing example?

"The assistant route is tested with simulated provider behavior. I can prove that invalid input does not reach Groq and that provider diagnostics do not leak the test secret. Those tests are fast and repeatable; live integration and browser tests would be additional layers."

A useful answer structure: decision -> reason -> example from the code -> limitation. For example: server fetch -> hide key -> /api/astrobot -> still needs rate limiting.

FIELD GUIDE / 15

Tough questions and honest limits

Knowing a limitation is stronger than defending a claim the code cannot support.

13. Is the entire app AI-powered?

"Only the assistant calls a language model. Brief summaries, tags, gallery filtering, and observing advice use deterministic rules. I have integrated a hosted model, not trained a new model or built a retrieval system."

14. Can the assistant answer a follow-up like "Why is that?"

"It may miss the context because the route sends only the newest question. To support follow-ups I would send a bounded conversation history, enforce role and length validation on the server, and manage token cost. Persistent history would be a separate feature."

15. What known issue would you fix first?

"Data provenance would be a strong first fix. The Moon service can combine a live daily response with sample upcoming phases without marking the overall result as fallback. I would return separate status and observation times for each part and test partial failures."

16. How trustworthy are the asteroid risk labels?

"They are simple display rules, not collision probabilities. The JPL mapping currently defaults the hazard flag to false, so it loses an unknown state. I would preserve unknown classification, revise overly certain wording, and keep official-source links prominent."

17. What is another concrete improvement?

"Brief links depend on the current feed and an ID that includes feed position. I would derive identity from a stable publisher identifier or canonical URL and persist an archive. Gallery partial-source warnings should also be surfaced more clearly."

Claims to avoid

Do not claim continuous real-time updates, a personalized Moon location, a custom-trained model, guaranteed hallucination prevention, comprehensive automated coverage, measured scalability, or a live physically accurate Earth. Also do not say there is no Three.js or no test suite: both exist in the current code.

WHEN YOU DO NOT KNOW OR HAVE NOT BUILT SOMETHING

"I have not implemented that yet. The current version does X. My next step would be Y, and I would verify it with Z."

Your demo and preparation plan

Use the interface to support the explanation, not replace it.

Time	Action	What to say
0:00-0:30	Open Discover	State the problem, audience, and value. Explain that the Earth is an illustration.
0:30-1:15	Open Briefs	Search a topic, open one item, show its publisher link. Explain normalization.
1:15-1:50	Open Moon or Asteroids	Name the provider. Explain location/time zone or units and label limits.
1:50-2:25	Open Gallery	Filter and open a lightbox. Explain source quality and progressive reveal.
2:25-3:00	Open Ask Astroboat	Ask a self-contained concept question. Explain server key protection and single-turn behavior.
3:00-3:30	Close with one tradeoff	Mention one implemented reliability choice and one concrete improvement.

Before the interview

Open the pages you plan to show and check the visible dates and source status. Confirm the assistant is configured if you intend to demo it. Prepare screenshots for an internet outage, labeled as screenshots rather than live data. If a provider fails during the demo, explain the fallback honestly and continue.

A focused 90-minute study session

- 0-15 min: practice the opening and draw the architecture from memory.
- 15-40 min: trace Briefs and the assistant in the actual files.
- 40-55 min: learn the Moon, asteroid, and 3D scene caveats.
- 55-75 min: answer questions 1-17 out loud without reading.
- 75-90 min: run the demo once and revise the cheat sheet.

Make your ownership story specific

Write four short answers: **What did I personally build? Where did I use AI, references, or help? What did I change after testing or feedback? What can I now explain or modify independently?** Use one real example for each. This guide cannot establish who authored each part or your development history.

PERSONALIZE THIS; DO NOT INVENT A STORY

"I used [actual tools/help] for [specific work]. I personally handled [specific contribution]. One issue I investigated was [real issue], and the result was [what changed]."

FIELD GUIDE / 17

The last-minute cheat sheet

Read this before you enter. Explain the chain rather than reciting jargon.

THE PROJECT

Astroboat = astronomy discovery + normalized public data + interactive tools + a server-proxied AI assistant.

Remember	Say it simply
Architecture	Next.js page -> service -> provider -> normalized data -> client UI.
Main stack	Next.js, React, TypeScript, Tailwind; XML parsing; Three.js; Groq.
Briefs	Nine feeds; rule-based summaries; dedupe; newest 40; 1-hour cache policy.
Gallery	Multiple official sources; metadata heuristics; allSettled; client filtering.
Moon	USNO; fixed Ahmedabad location; UTC phase events; partial-status gap.
Asteroids	JPL -> optional NeoWs -> samples; units converted; no impact prediction.
AI	Only newest question; server key; 1,000-character cap; 30-second timeout; no streaming or retrieval.
3D Earth	Lazy-loaded Three.js; decorative; pause when hidden; dispose resources.
Persistence	No application database. Chat state resets on page reload.
Verification	15 assistant tests passed; TypeScript passed; production build/live demo not verified in this review.

Five things you should be able to do unaided

- 1. Give a 45-second pitch without listing every library.
- 2. Draw the architecture and trace one request.
- 3. Explain one failure path and one security decision.
- 4. Name two limitations and propose realistic fixes.
- 5. Explain your own contribution with a real example.

If you get stuck: "Let me trace the flow." Start with what the user does, then where the request goes, how the data is transformed, and what comes back. It is fine to open the relevant file and reason from it.

Code map and evidence

Use this page to find the implementation behind an answer.

Paths below are relative to the project root:
/Users/khushaldangar/Desktop/astrobot-v2

Ref	Files / entry points	What they establish
S1	package.json tsconfig.json	Declared stack, scripts, strict TypeScript configuration.
S2	app/page.tsx; app/layout.tsx components/home/Hero.tsx	Homepage composition, shared layout, live-service calls, Earth import.
S3	services/briefs-service.ts getAstronomyBriefs; mapFeedItemToBrief	Feed pipeline, summary rules, deduplication, status, image enrichment.
S4	app/briefs/page.tsx app/briefs/[slug]/page.tsx components/briefs/BriefsClient.tsx	Server-to-client flow, local search, detail lookup and 404 behavior.
S5	services/gallery-service.ts getGalleryData; scoreGalleryCandidate	Concurrent providers, scoring, ranking, empty fallback.
S6	app/api/gallery/route.ts components/gallery/GalleryClient.tsx	API slicing versus local UI reveal and image query parameter.
S7	services/moon-service.ts app/moon/page.tsx	USNO requests, fixed default usage, phase normalization, fallback gap.
S8	services/events-service.ts services/asteroids-service.ts lib/constants.ts	Provider chains, cooldown, units, risk rules, cache intervals and limits.
S9	app/api/astrobot/route.ts components/ask/AstrobotClient.tsx	Validation, prompt, default model, errors, single-turn chat state.
S10	components/visuals/EarthScene.tsx components/visuals/earth-scene.ts	Lazy loading, WebGL, shaders, visibility, motion, and cleanup.
S11	tests/astrobot-route.test.cjs	The 15 deterministic assistant tests run for this guide.
S12	app/articles/page.tsx; app/learn/page.tsx app/satellites/page.tsx lib/search-index.ts	Paused routes; seven searchable destinations and six quick links.

External references, checked 29 September 2026: [NASA Moon Phases](#) for lunar concepts; [JPL CAD API documentation](#) for response fields and units. All project-specific claims come from the local code and stated checks. Suggested improvements are proposals, not implemented features.